



Literature review of Application of AI in improving gaming experience: NPC behavior and dynamic difficulty adjustment

Lappeenranta–Lahti University of Technology LUT
Master's Programme in Software Engineering and Digital Transformation
2024

Shuao Chen

Examiner(s): Associate professor Jussi Kasurinen

Associate professor Antti Knutas

ABSTRACT

Lappeenranta–Lahti University of Technology LUT

LUT School of Engineering Sciences

Master's program in Software Engineering and Digital Transformation

Shuao Chen

Literature review of Application of AI in improving gaming experience: NPC behavior and dynamic difficulty adjustment

Master's thesis

2024

53 pages, 8 figures, 2 tables and 1 appendices

Examiner(s): Associate professor Jussi Kasurinen

Associate professor Antti Knutas

Keywords: Artificial intelligence, Non-player character behavior, Dynamic Difficulty Adjustment, Game.

The application of artificial intelligence (AI) in the game industry is deepening, bringing more immersive and personalized experience to player. This systematic literature review focuses on the application of AI in enhancing non-player character (NPC) behavior and dynamic difficulty adjustment (DDA). Through a comprehensive review and analysis of existing research, we found that AI has made significant progress in giving NPCs more intelligent, realistic, and diverse behaviors. Meanwhile, AI-driven DDA mechanisms can also adjust game difficulty in real time according to player's skill levels, thus maintaining player challenge and engagement.

ACKNOWLEDGEMENTS

I would like to express our sincere gratitude to our supervisors, whose guidance and support have been invaluable throughout this research journey. Their expertise, encouragement, and constructive feedback have played a pivotal role in shaping this study and enriching our understanding of the subject matter. Additionally, I am deeply grateful to our families and friends for their unwavering encouragement and understanding, which have sustained us during this endeavor.

Shuao Chen

Table of contents

Abstract

(Acknowledgements)

1	<i>Introduction</i>	7
1.1	Research Questions and Objectives	9
1.2	Scope and Methodology	9
1.3	Significance of the Study	10
1.4	Structure of this thesis	10
2	<i>Background and Related Work</i>	11
2.1	AI Applications in Games: An Overview	11
2.2	NPCs (Non-player character) Behavior	13
2.3	Traditional Approaches of NPCs Behavior	14
2.3.1	Rule-Based Systems	14
2.3.2	State Machines	17
2.3.3	Behavior Tree	19
2.3.4	Shortcomings in Traditional AI	21
3	<i>Literature Search and Selection</i>	23
3.1	Databases and Keywords	23
3.2	Literature Screening and Selection Process	25
3.2.1	Initial Screening	25
3.2.2	Abstract Screening	26
3.2.3	Full-Text Review	26
3.2.4	Data Extraction Process	27
4	<i>Result</i>	27
5	<i>AI Applications in NPC Behavior</i>	29
5.1	Real AI in NPCs Behaviors	30
5.1.1	Real AI vs Traditional AI	31
5.1.2	Deep Q-network (DQN)	32
5.1.3	Alpha Go	33
5.1.4	Alpha Star	34
5.1.5	OpenAI Five	35
5.2	Dynamic Difficulty Adjustment	36
5.3	Hidden Markov Models	40

5.4	Reinforcement learning	43
6	<i>DISCUSSION AND FUTURE WORK</i>	46
6.1	Research Gaps and Challenges.....	46
6.2	Future Research	47
7	<i>Conclusion</i>	48
	<i>References</i>	50

Appendices

Appendix 1. Table of the selected papers.

LIST OF SYMBOLS AND ABBREVIATIONS

AI	Artificial Intelligence
DDA	Dynamic Difficulty Adjustment
NPC	Non-Player Character
FSM	Finite State Machine
FSA	Finite State Automaton
BT	Behavior Tree
DRL	Deep Reinforcement Learning
RL	Reinforcement Learning
MPRL	Multiple-Periodic Reinforcement Learning)
SEC	Skilled Experience Catalogue
HMM	Hidden Markov Model
NLP	Natural Language Processing
RTS	Real-Time Strategy
RPG	Role-Playing Game
MOBA	Multiplayer Online Battle Arena

1 Introduction

In today's digital entertainment era, video games have become a popular form of entertainment, attracting billions of players worldwide. Games and artificial intelligence have a long history together (Yannakakis & Togelius, 2018). As artificial intelligence (AI) and games intersect, games have completely changed how players interact with and experience video games. Artificial intelligence (AI) technology is increasingly incorporated into game development to enhance player engagement, personalize gaming experience, and create dynamic and immersive virtual worlds. This paper focuses on two key applications of Artificial intelligence (AI) in enhancing gaming experiences: non-player character (NPC) behavior modelling and dynamic difficulty adjustment (DDA). It aims to deeply study their technical principles, implementation methods, and far-reaching impact on the gaming industry.

Video games produce reactionary, resilient, or clever behavior, mostly on non-player characters (NPCs), who resemble Artificial Intelligence (AI) (U, et al., 2021). Traditional NPCs are often limited by preset scripts, with monotonous and mechanical behavior patterns, making it difficult to engage in meaningful interactions with players. However, the introduction of AI technology has injected new vitality into NPC behavior. AI-driven NPCs can dynamically respond to player actions and the environment, exhibiting emotions, learning abilities, and autonomous decision-making, thereby creating a more immersive and believable game world. Through advanced algorithms like machine learning and natural language processing, developers can build more intelligent and personalized NPCs.

At the core of gaming experience lie the gameplay mechanics, comprising the rules, controls, challenges, and objectives that structure player interaction with the game system. Smooth mechanics facilitate a sense of agency and mastery, while well-balanced challenges sustain player motivation and flow state. Koster Raph (2013) quoted the flow (figure 1) of (Csikszentmihalyi, 2008) and has outlined various situations that may lead players to experience boredom in games. For instance, when a game is excessively easy or challenging, when players cannot discern patterns within the game, when the pace is too slow and feels repetitive, or when the pace is too fast, making it challenging for players to keep up. These

situations are more or less related to the difficulty of the game. If one desires to keep players consistently enjoying the game, controlling the difficulty is a crucial aspect. However, balancing a digital game is one of the most challenging and time-consuming tasks during game development. Due to the subjectivity attached to this process, the game difficult may be pleasant for some players while unpleasant for others due to each individual perception (Zamith, et al., 2020).

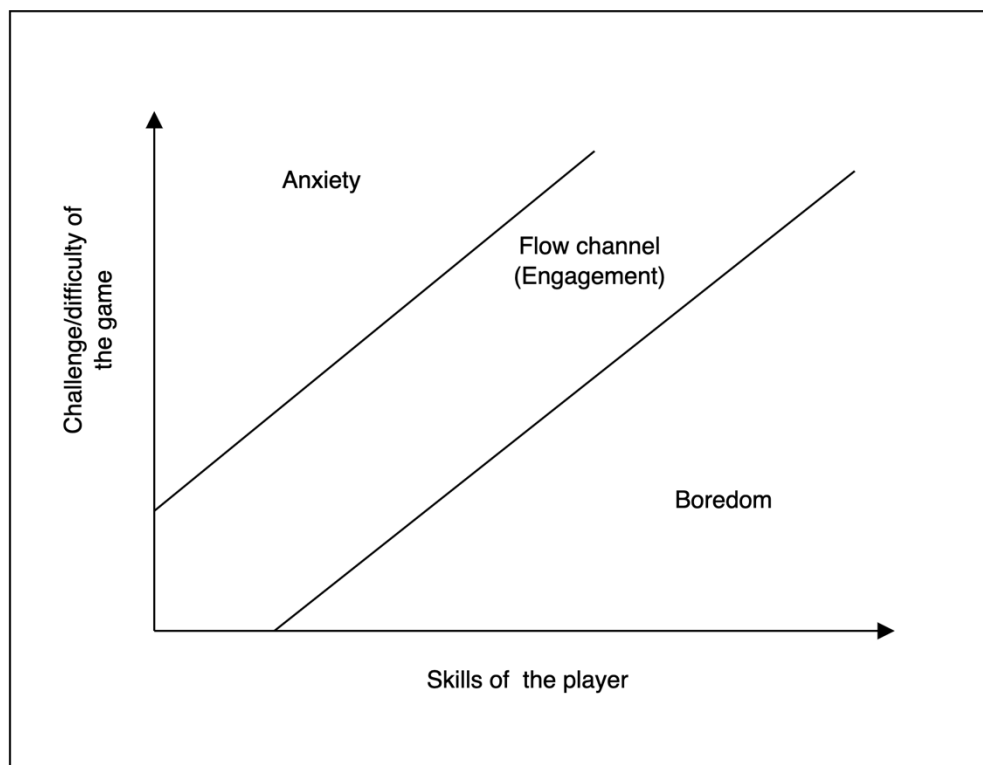


Figure 1 Flow Channel

Meanwhile, DDA has always been a challenge in game design. Balancing the challenge and playability of a game to ensure that players of different skill levels can have the best gaming experience is a long-standing challenge for developers. The MDA framework proposed by Hunicke et al. (2004) emphasizes the interaction between "mechanics," "dynamics," and "aesthetics" in games, and the emergence of AI provides new ideas for achieving dynamic and adaptive game experiences. By analysing player behavior data, AI algorithms can adjust game difficulty in real-time, ensuring that each player experiences a "just right" challenge, thereby maximizing the game's fun and engagement (Cherukuri & Glavin, 2022).

This thesis will first review the development of AI in the gaming industry, from early expert systems to current deep reinforcement learning models, showcasing the evolution of AI technology in game design. Subsequently, this thesis will delve into the specific applications of AI in NPC behavior modeling and DDA, including key technologies such as behavior trees, Markov decision processes, and emotion modeling, and analyze their effects through practical case studies. Finally, this thesis will look ahead to the future development trends of AI in the gaming field, exploring the opportunities and challenges it may bring.

1.1 Research Questions and Objectives

This systematic literature review seeks to explore the application of AI in enhancing NPC behavior and implementing DDA to improve the overall gaming experience. The primary research questions guiding this review are:

RQ-1: How has AI been applied to enhance NPC behavior in video games?

RQ-2: What are the main AI techniques used for Dynamic Difficulty Adjustment in gaming?

RQ-3: How do these AI applications impact player engagement, satisfaction, and retention?

The objectives of this review are to:

- Provide a comprehensive overview of current AI techniques used in NPC behavior and DDA.
- Analyze the effectiveness of these AI applications in improving player experience.
- Identify the challenges and limitations associated with AI-driven NPCs and DDA systems.
- Suggest potential directions for future research and development in this area.

1.2 Scope and Methodology

This review focuses on academic literature and industry reports published in the last decade (2014-2024) to ensure the inclusion of the most recent and relevant advancements in AI

applications in gaming. The review process involved a systematic search of several academic databases, including IEEE Xplore, Web of Science, ACM Digital Library, and Google Scholar, using keywords such as “AI in gaming,” “NPC behavior,” “game experience” and “dynamic difficulty adjustment.”

The selection criteria for the literature included peer-reviewed journal articles, conference papers, and significant industry reports. Studies were excluded if they did not directly address AI applications in NPC behavior or DDA or were not available in English. The final set of papers was analyzed to extract information on AI techniques used, their implementation in games, and the outcomes related to player experience.

1.3 Significance of the Study

The significance of this study lies in its potential to synthesize existing knowledge on the role of AI in gaming, particularly in enhancing NPC behavior and implementing DDA. By systematically reviewing and analyzing current research, this study aims to provide valuable insights for game developers, researchers, and AI practitioners. Understanding the state-of-the-art AI applications can guide the development of more immersive and enjoyable gaming experiences. Furthermore, identifying gaps and challenges in the current literature can inform future research directions and technological advancements.

1.4 Structure of this thesis

The remainder of this thesis is structured as follows:

- Section 2: Background about AI and NPC behavior in game.
- Section 3: Literature Search and Selection outlines the databases, search terms, and criteria used to identify relevant studies.
- Section 3: AI in NPC Behavior discusses the various AI techniques applied to NPC behavior.
- Section 4: AI in Dynamic Difficulty Adjustment covers the AI methods used in DDA, recent advancements, and case studies from actual games

- Section 5: Integrated Discussion synthesizes findings from the previous sections, highlighting the interplay between NPC behavior and DDA, and discussing the broader implications for game design.
- Section 6: Conclusion summarizes the key insights from the review and proposes future research directions.

By following this structure, this review aims to present a thorough and coherent analysis of how AI can enhance the gaming experience through improved NPC behavior and dynamic difficulty adjustments.

2 Background and Related Work

2.1 AI Applications in Games: An Overview

Game intelligence has witnessed a remarkable evolution since its inception. The history of game intelligence can be traced back to the earliest video games about, where simple rule-based algorithms were used to control non-player characters (NPCs) or opponents. Historically, this has been the first and, for a long time, the only approach to using AI in games (Yannakakis & Togelius, 2018). Early game AI primarily relied on rudimentary rule engines and finite state machines to govern enemy behavior and level generation. Pac-Man [Midway Games West, Inc., 1979] was the first game many people remember playing with fledgling AI (Millington, 2019) . It featured rudimentary AI behaviors, mainly focused on basic movement patterns and simple decision-making processes. These early games laid the groundwork for future developments in game intelligence.

Game AI didn't change much until the mid-1990s, most computer-controlled characters prior to then were about as sophisticated as a Pac-Man ghost (Millington, 2019). The 1990s marked a significant milestone in the field of game intelligence, with the emergence of more complex game genres such as real-time strategy (RTS) and role-playing games (RPGs). Games like "Warcraft" and "Final Fantasy" introduced AI-driven characters with more

advanced behaviors, including pathfinding, resource management, and strategic decision-making.

The turn of the millennium saw the rise of 3D graphics and immersive gaming environments, further propelling advancements in game intelligence. Titles like "Half-Life" and "The Sims" introduced AI-driven characters with more nuanced behaviors, enabling interactive storytelling and dynamic player interactions.

In recent years, with the advent of machine learning and deep learning technologies, game intelligence has reached new heights of sophistication. Modern games leverage AI algorithms for procedural content generation, adaptive difficulty adjustment, and personalized player experiences. Titles like "The Elder Scrolls V: Skyrim" and "Horizon Zero Dawn" showcase the capabilities of AI-driven game design in creating dynamic and immersive worlds.

While the focus thus far has been on enhancing player experiences, the evolution of AI in games extends beyond traditional gaming experiences. Most early research on game-playing AI was focused on classic board games, such as Checkers and Chess (Yannakakis & Togelius, 2018).

In the mid-20th century, Claude Shannon's research laid the foundation for computer game theory and provided the basis for the development of chess artificial intelligence. In the late 1950s, Arthur Samuel developed the first learning algorithm that allowed computers to improve themselves in the game of chess. From the 1960s to the 1970s, pioneers such as Harlan D. Mills and Richard Michael Fikes promoted the application of heuristic programming and provided important support for the development of chess artificial intelligence.

In 1997, IBM's Deep Blue defeated world chess champion Garry Kasparov, demonstrating the importance of deep search and computing power. Subsequently, heuristic methods such as Monte Carlo tree search further improved computer performance in the complex game of chess. Entering the 21st century, the rise of machine learning and deep learning has brought revolutionary changes to game AI. By learning from massive amounts of data, AI systems

can autonomously optimize decisions and exhibit intelligence closer to human levels. In 2016, AlphaGo defeated Go world champion Lee Sedol, demonstrating the potential of deep learning in the field of chess artificial intelligence. Just one year later, AlphaGo Zero's self-playing learning further improved the capabilities of Go artificial intelligence, highlighting the huge potential of deep learning in the field of chess artificial intelligence.

2.2 NPCs (Non-player character) Behavior

Non-Player Characters, commonly known as NPCs, play a crucial role in the dynamics of video games and simulations, aiming to create NPCs that exhibit believable, engaging, and diverse actions to enhance player experiences. NPC Behavior refers to the set of rules, algorithms, and responses programmed into these characters to simulate human-like actions and interactions within a digital space. From simple routines like walking and talking to complex decision-making in response to changing game conditions, NPC Behavior is a cornerstone in the realm of software engineering, contributing significantly to the overall user experience. Over the years, various approaches have been explored, ranging from traditional rule-based systems to cutting-edge AI techniques.

Activities associated with Artificial Intelligence (AI) in computer games are designed for behaviors of Non-Player Characters (NPC) (Sekhavat, 2017). Traditionally, NPC behaviors were scripted or relied on simple rules (Yannakakis & Togelius, 2018). Early NPC behaviors were primarily governed by rule-based system, finite state machines (FSMs) and Scripting. These systems are relatively simple to implement and understand, making them a popular choice for early game AI. However, their rigidity and inability to adapt to dynamic game environments often lead to repetitive and predictable NPC behaviors.

FSMs represent NPC behavior as a collection of discrete states, with transitions between states triggered by specific events or conditions. Scripting allows for more flexibility and expressiveness compared to FSMs, as it enables designers to write custom code to control. However, scripting can quickly become unwieldy for complex behaviors, leading to difficulties in maintenance and debugging.

While rule-based systems and FSMs have laid the groundwork for NPC behavior modeling, their inherent limitations in terms of flexibility, scalability, and adaptability have spurred the exploration of more sophisticated AI-driven approaches.

The main advantage of BTs compared to FSMs is their modularity, and Behavior Tree is flexibility than FSM (Colledanchise & Ogren, 2014). BTs represent NPC behavior as a hierarchical tree structure, where each node represents a task or condition. The tree is traversed from the root node, and the execution of each node depends on the outcome of its child nodes. This hierarchical structure allows for more complex and varied behaviors compared to FSMs. However, designing and maintaining complex BTs can be challenging, especially for large-scale games with numerous NPCs.

2.3 Traditional Approaches of NPCs Behavior

Various approaches are proposed to modelling NPCs behavior, including Rule-Based Systems, Finite State Machines (FSMs), and Behavior Trees. These approaches provide structured frameworks for defining and managing the states, transitions, and decision-making processes that govern NPC actions and responses, significantly influencing the player experience.

2.3.1 Rule-Based Systems

Rule-based systems are the simplest and fairly limited to control the behavior, in which the behavior of an NPC is a simple function of present conditions (Sekhavat, 2017).

The Rule-based system emerged within the field of artificial intelligence in 1970s. Ananta Srisuphab (2013) highlighted three primary components essential to the rule-based system: a rule base, a knowledge body or facts, and an inference engine. Sekhavat (2017) emphasized that a rule-based system consists of a knowledge-base and a set of if-then rules. They share a commonality of three main components, each playing a crucial role in the decision-making process:

- **Rule Base:** This component is a specific type of knowledge base and serves as the foundation of the rule-based system, consisting of a set of if-then rules that define the behavior of the NPC. Each rule specifies a condition (if) and an associated action (then), dictating how the NPC should respond to different situations encountered in the game environment.
- **Inference engine:** This component is responsible for interpreting and applying the rules defined in the system. It evaluates the current state of the game world against the conditions specified in the rules and triggers the corresponding actions based on the match between conditions and the state of the environment.
- **Facts:** also known as working memory or short-term memory, acts as a storage mechanism for holding information relevant to the decision-making process. It temporarily retains data about the current state of the game world, such as the NPC's surroundings, the player's actions, and other contextual factors necessary for rule evaluation.

The system cycles through a match-resolve-act loop. First, the condition sides of all rules are matched against the contents of facts. The collection of satisfied rules is called a "conflict set." Once all the rules that match the current state are gathered, the system needs to choose one to execute in order to resolve the conflict. In the "act" phase, the action of the rule selected in the resolve step will be executed. After the execution, the system cycles return to match phase.

These components work together synergistically to enable the rule-based system to govern NPC behavior effectively within the game environment. While rule-based systems offer simplicity and ease of implementation, they are limited in their ability to capture the complexity and nuance of human-like behavior. Despite their limitations, rule-based systems remain a valuable tool in game development, particularly for scenarios where deterministic, rule-driven behavior is sufficient to achieve the desired gameplay experience.

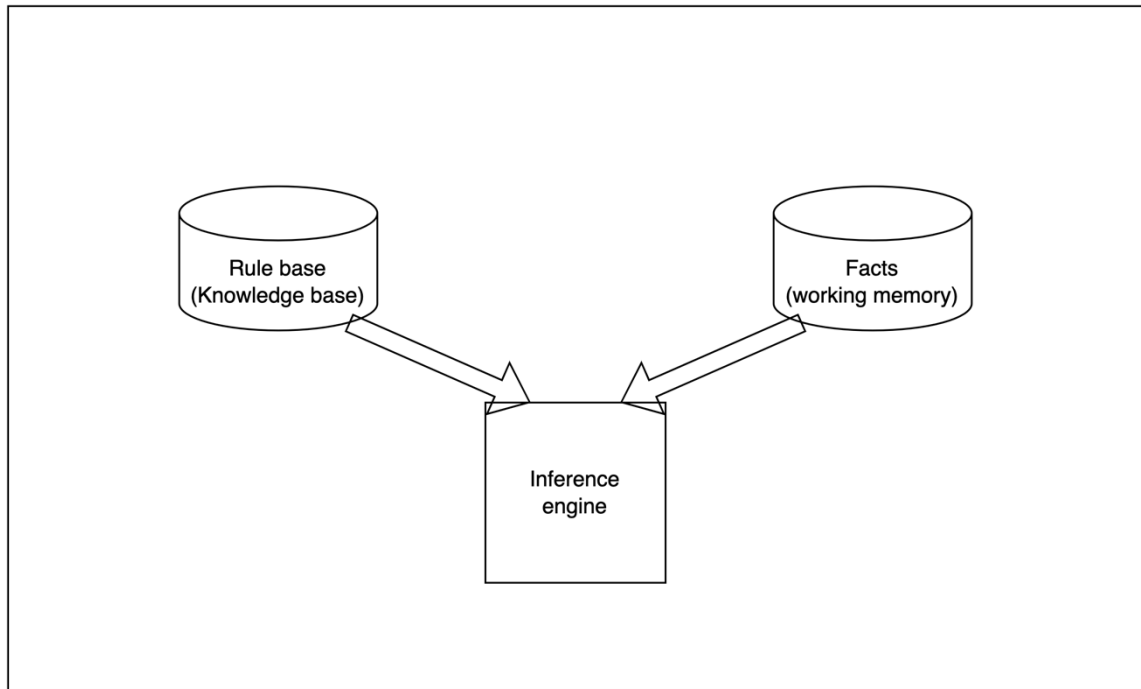


Figure 2 Rule-Based System

Kirby (2011) summarizes the advantages and disadvantages of rule-based system as follow:

Advantages:

- Effectively manages complexity without the constraints of state-based approaches.
- Can imitate human behavior when rules are derived from human play.
- Exhibits expert-level performance when rules are based on high expertise.
- Easier for novice AI programmers to get started.
- Flexibility in the complexity of rule execution.

Disadvantages:

- Challenges in crafting effective rules for every situation.
- Requires domain expertise to write rules.
- No inherent constraints on rule execution, leading to potential complexity.
- As the complexity of the system increases, the cost of adding new rules each time will prompt.

2.3.2 State Machines

State machine, generally referring to a Finite State Machine (FSM), also known as a Finite State Automaton (FSA), is a mathematical computational model that represents a finite number of states along with transitions and actions between these states. FSM has been there in gaming industry for a long time and is still being used in today's modern games by many developers. Creating AI agents using FSM which act like Simple reflex agent makes game more interactive and fun to play (Jagdale, 2021).

An FSM is a directed graph in which each node represents a state. An NPC can only be in one of the states. An edge in this graph corresponds to an event that triggers a change from one state to another state. Unlike rule-based systems, the behavior of an NPC is a function of the current state of the NPC as well as the present conditions (Sekhavat, 2017). This design allows for the simulation of more complex behaviors in games, enabling NPCs to respond differently based on the surrounding environment and specific situations. Here's a brief overview of how FSMs are applied in gaming:

- **States:** Game characters can exist in different states, such as idle, walking, running, attacking, or resting. Each state represents a specific behavior or activity.
- **Transitions:** Transitions define the conditions of how a character moves from one state to another. For instance, a character may transition from the idle state to the walking state when the player presses the movement key.
- **Actions:** Operations performed when a state transition occurs. These actions can involve updating the system state, generating outputs, or any other relevant behavior. For example, when the player presses the movement key, adjust the animation based on the current state.

Today's games are complex games that require a large number of states (Sekhavat, 2017). FSMs can be organized hierarchically, allowing for a more modular and scalable approach. Sub-states can represent detailed behaviors within a broader state. Figure 5 shows an example for Hierarchical FSM.

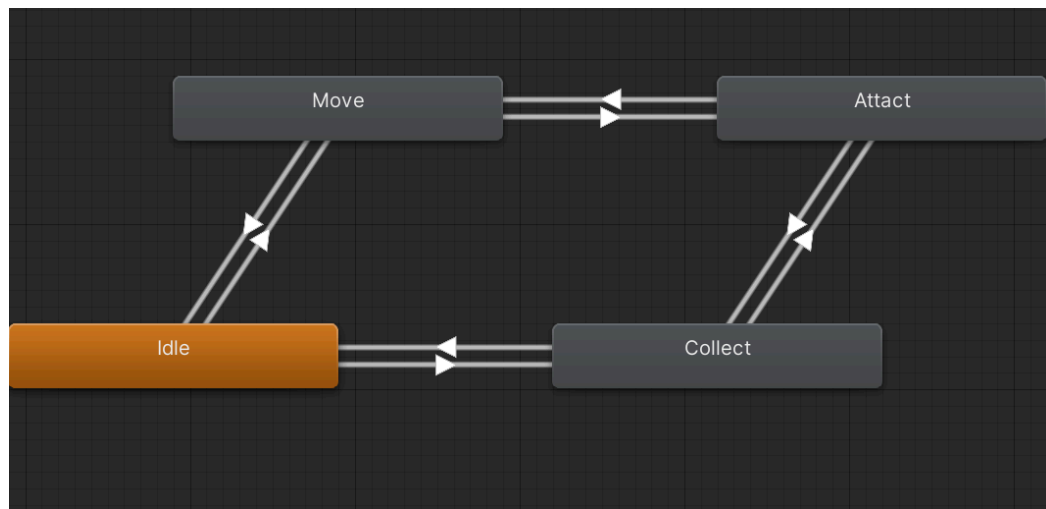


Figure 3 State Machine in Unity

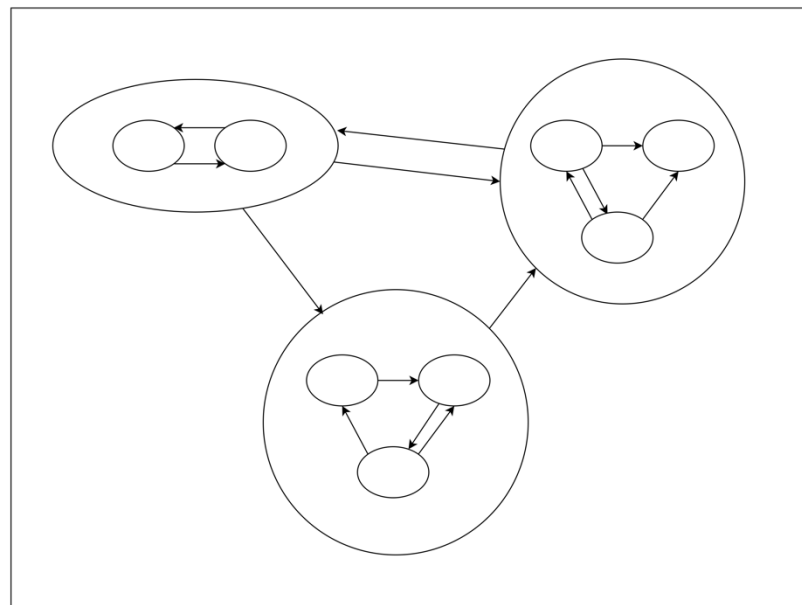


Figure 4 Hierarchical FSM

By employing FSM, developers can create diverse, predictable, and engaging NPC behaviors. For instance, an NPC might be in an exploration state, transitioning to an attack state when an enemy is detected, or switching to an evasion state when faced with a scarcity of resources. This state transition model enhances the gaming experience, providing characters with a more realistic and challenging aspect. Games like PACMAN or Half Life implemented FSM as AI agents in their games as enemies. Many games Call of Duty and Tomb Raider have successfully incorporated the FSM AI design. If we try to capture

maximum range of Behaviors that our NPC can encounter, we can create a successful FSM AI design (Sekhavat, 2017).

FSMs are incredibly simple to design, implement, visualize, and debug. Further they have proven they work well with games over the years of their co-existence (Yannakakis & Togelius, 2018). However, FSMs have some drawbacks that can pose challenges in certain scenarios:

- State explosion: In complex systems, the number of states may become very large, leading to the state explosion problem. This increases the complexity and maintenance cost of the system, reducing its scalability.
- Limited expressive power: FSMs are suitable for describing systems with relatively simple states and transitions. For complex systems, their expressive power is limited, and more sophisticated models may be required.
- Difficulty in maintenance: As the complexity of the system increases, maintaining FSMs may become challenging. Adding new states or transitions may affect the existing system behavior, leading to unpredictable consequences.

Despite these limitations, FSMs remain effective tools in many applications. In practice, developers may opt for different modelling tools and techniques to better meet the specific requirements of a given system.

2.3.3 Behavior Tree

Behavior Tree (BT) is a widely used technique in artificial intelligence and game development for modelling and controlling the behavior of characters or entities. It provides a hierarchical and modular approach to defining and organizing the decision-making process of an agent. An agent running a behavior tree performs a depth-first search on the tree to find and run the lowest level leaf node (Sekhavat, 2017).

The main advantage of BTs compared to FSMs is their modularity (Colledanchise & Ogren, 2014), and Behavior Tree is flexibility than FSM. Behavior tree allows hierarchical FSM, which introduces another layer of decisions. So, the bot decides among branches of

behaviors that define the state it is in (Newton & Feng, 2016). Both the Unity Engine and Unreal Engine provide the editors for uses to create the Behavior Tree. These editors enable game developers to design and manage complex behavior logic for controlling characters, NPCs, and other entities in the game.

Let's take a look the concepts of Behavior Trees using examples and features found in the Unreal Engine.

There are components found withing Unreal:

- **Root Node:** A root node is the starting point for a behavior tree. Every behavior tree has only one Root Node. Besides, it can only establish a connection with one node, and this node cannot be a Service Node and Decorator Node.
- **Composite Node:** Composite nodes serve as the root for branches within a Behavior Tree, establishing fundamental rules for branch execution. They support the application of Decorators to alter entry into the branch or terminate execution midway. Additionally, Services can be linked to them, becoming active solely when the Composite's children are being executed.
- **Decorator nodes:** Decorator nodes are used to add conditional checks or modifications to the execution of child nodes within the tree. They evaluate conditions and base on the result, they either allow the execution to continue or cancel it.
- **Services node:** Services node is a special type of node that performs tasks or updates information while the behavior tree is running. Unlike other nodes that execute only once or based on specific conditions, Services nodes continuously run in the background while their parent Composite node is active.
- **Task node:** Task node represents a specific action or task that an AI-controlled character can perform. Task nodes are used to encapsulate individual actions or behaviors, such as moving to a location, attacking an enemy, or interacting with objects in the game world.
- **Blackboard:** The Blackboard is a data storage system used by the Behavior Tree to store and share information between nodes.

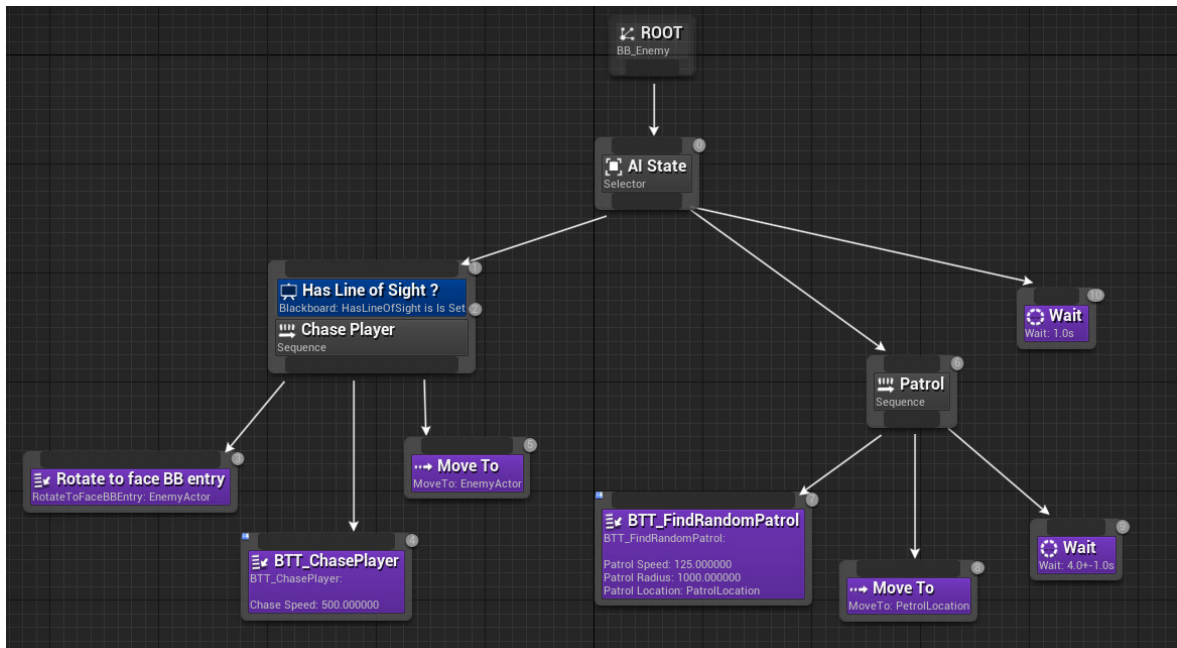


Figure 5 Behavior in Unreal Engine

Overall, the combination of these features and components makes Behavior Trees a powerful tool for modelling and controlling complex behaviors in AI and game development, offering modularity, flexibility, and scalability than FSMs.

2.3.4 Shortcomings in Traditional AI

Traditional AI in gaming, while groundbreaking in its time, exhibited several significant shortcomings that detracted from the overall player experience. From resorting to cheating mechanisms to its inability to adapt and learn, traditional AI often fell short in providing players with a truly engaging and fair gaming environment. This was particularly evident in early real-time strategy (RTS) games like "Command & Conquer: Red Alert" and "StarCraft", where players frequently encountered predictable AI behavior and exploitable vulnerabilities. This section delves into the specific shortcomings of traditional AI, highlighting its impact on gameplay and player satisfaction.

- **Cheating:** In traditional AI systems, as players choose the higher difficulty enemy, the AI often resorts to various forms of cheating. For example, the AI might receive more initial resources and greater resource acquisition bonuses. In games like "Command & Conquer: Red Alert", at the "Crazy" difficulty level, the AI can amass a significantly

larger army of soldiers and tanks within the first few minutes, launching devastating attacks on the player. If a player faces multiple enemies at this difficulty level, survival becomes nearly impossible, as the AI opponents, even those not allied, will prioritize cooperating to eliminate the player. Meanwhile, the AI ignores the fog of war to a certain extent, and automatically moves to the player's base without exploring the map.

- **Vulnerabilities in traditional AI:** since the AI's scripts are pre-set by developers, it lacks the ability to learn and improve on its own. Once players discover these AI vulnerabilities, they can exploit them repeatedly to defeat the AI. For instance, in "Command & Conquer: Red Alert", one way to defeat multiple insane AI opponents is by taking advantage of the AI's inability to build naval and air forces. Players can construct a shipyard before being overwhelmed and produce a fleet with a range advantage and the ability to carry combat units. By attacking the enemies from the sea, players can steadily erode the AI's forces. Moreover, at a certain point, the AI stops producing and constructing any units, allowing players to slowly and steadily wear down the enemy.
- **Predictability:** AI enemies follow a pre-written build order to develop their bases and engage in combat using fixed tactics. This behavior pattern makes the AI very predictable and easy for players to exploit. For example, in StarCraft, the AI often attacks at the same time and place, allowing players to prepare in advance and easily counter the attacks.

In Multiplayer Online Battle Arena (MOBA) games, traditional AI also exhibits above shortcomings. In "Dota", traditional AI characters often follow fixed paths and execute similar actions at specific times. For example, traditional AI may attempt to secure in-game runes at set intervals without considering the current game state or enemy positions. Additionally, they may lack awareness of controlling map vision, leading to frequent ambushes by enemy forces.

In terms of skill and item selection, traditional AI also demonstrates relative rigidity. For instance, they may consistently follow the same skill upgrade path to level up their abilities, without considering the compositions of opponents or the current game situation. Similarly,

when purchasing items, traditional AI may tend to acquire the same set of items rather than making flexible choices based on specific circumstances.

Furthermore, traditional AI also exhibits clear shortcomings in teamwork. They may lack the ability to collaborate with teammates effectively, making it difficult to execute team strategies or coordinate with other AI allies. For example, even when teamwork could lead to victory, traditional AI may opt to act independently, resulting in tactical inconsistencies and inefficiencies.

It's worth noting that traditional AI typically fails to leverage terrain features, such as high and low ground affecting miss chances or shadows in the forest providing cover, which players can exploit to gain advantages in combat. This allows players to gain an edge by strategically utilizing environmental factors, while traditional AI cannot utilize these terrain features.

Overall, the NPCs controlled by traditional AI often exhibit rigid behavioral patterns, lack adaptability, creativity, and learning capabilities. These NPCs in games typically execute pre-programmed fixed instructions, unable to adjust or improve their behavior flexibly based on circumstances. This restrictiveness results in a relatively shallow and less challenging gaming experience, as players can easily predict and exploit the NPCs' behavioral patterns. Moreover, AI cheating will make players feel unfair, and players will feel that they are not defeated by the opponent's operation and strategy, which brings a very unpleasant experience to players.

3 Literature Search and Selection

3.1 Databases and Keywords

The foundation of a robust systematic literature review lies in a thorough and systematic search of relevant literature. For this review, several academic databases were identified as the primary sources for collecting pertinent research articles. The chosen databases include:

- IEEE Xplore: Known for its extensive collection of high-quality research papers in the fields of computer science and engineering.
- ACM Digital Library: Offers a vast repository of scholarly articles and conference proceedings related to computing and information technology.
- Web of Science: A comprehensive and multidisciplinary research database that provides access to a vast repository of scholarly literature across various fields.

To ensure a focused and relevant search, a set of keywords and phrases were formulated. The keywords selected for the search process included:

- "AI"
- "Artificial Intelligence"
- "Reinforcement learning"
- "Machine learning"
- "Game"
- "Gaming"
- "NPC behavior"
- "non-player character behavior"
- "Dynamic Difficulty Adjustment"
- "DDA"
- "Dynamic Game Balancing"
- "Adaptive difficulty"

Boolean operators such as AND, OR, and NOT were used to combine keywords and refine search results. For example, the chosen search query was:

("AI" OR "Artificial Intelligence" OR "Reinforcement learning" OR "Machine learning")
AND ("gaming" OR "game") AND ("NPC behavior" OR "non-player character behavior"
OR "NPC" OR "dynamic difficulty adjustment" OR "adaptive difficulty" OR "Dynamic
Game Balancing" OR "DDA")

Table 1 Publications found using chosen search query

Databases	Total of articles found
IEEE Xplore	133
ACM Digital Library	519
Web of Science	101

A total of 753 publications were found in the three databases. Table 1 shows the number of articles found in each database.

3.2 Literature Screening and Selection Process

The literature search yielded a substantial number of articles across the selected databases. To systematically narrow down the results to the most relevant studies, a multi-stage screening process was employed, comprising initial screening, abstract screening, and full-text review.

3.2.1 Initial Screening

The initial screening involved scanning titles and keywords to eliminate obviously irrelevant articles. Inclusion and exclusion criteria were established to streamline this process:

- Inclusion Criteria:
 - Articles published between 2014 and 2024.
 - Peer-reviewed journal articles, books, conference papers, and significant industry reports.
 - Studies directly addressing AI applications in NPC behavior or dynamic difficulty adjustment.
 - Articles available in English.
- Exclusion Criteria:
 - Articles not related to gaming or AI.
 - Non-peer-reviewed sources, such as opinion pieces, editorials, and blog posts.
 - Studies focusing on unrelated aspects of gaming, such as hardware or network performance.

Through this initial screening, articles that clearly did not meet the inclusion criteria were excluded. This step reduced the number of articles from several thousand to a more manageable subset.

3.2.2 Abstract Screening

The next step involved a detailed review of the abstracts of the remaining articles. This phase aimed to further filter out studies that, despite having relevant titles and keywords, did not provide substantial contributions to the specific topics of AI in NPC behavior or DDA. During abstract screening, each abstract was evaluated for:

- Relevance to the research questions and objectives.
- Specific mention of AI techniques applied to NPC behavior or DDA.
- Indication of empirical results, case studies, or theoretical advancements in the field.

Articles that lacked sufficient relevance or did not offer substantial insights into the application of AI in the defined areas were excluded at this stage. This further refined the list to a selection of articles that warranted a full-text review.

3.2.3 Full-Text Review

The final stage involved a thorough review of the full texts of the selected articles. This comprehensive evaluation focused on extracting detailed information relevant to the research questions. Key aspects considered during the full-text review included:

- Detailed Descriptions of AI Techniques: An in-depth analysis of the AI methods and algorithms used in the studies.
- Application Context: Specific details about the game or simulation where the AI was implemented.
- Empirical Results and Findings: Key outcomes, including performance metrics, player feedback, and observed improvements.

- Challenges and Limitations: Issues encountered during the implementation of AI and areas needing further research.
- Conclusions and Future Directions: Authors' conclusions and recommendations for future work.

To ensure consistency and reliability, a standardized data extraction form was used to document key information from each article. The form included fields for bibliographic information, study aims, AI techniques employed, application context, results and findings, challenges and limitations, and conclusions and future directions.

3.2.4 Data Extraction Process

Data extraction was performed systematically to ensure all relevant information was captured accurately and consistently. The extraction form included the following fields:

- Bibliographic Information: Title, publication year.
- Techniques: Detailed descriptions of the AI methods and algorithms used.
- Research Topic: DDA or NPC behavior.

The table of data extraction can be found in the appendix 1.

4 Result

After completing the full-text review, a final selection of approximately 17 articles was identified as the core literature for this review. These articles provided a comprehensive and representative sample of the current research landscape on AI applications in NPC behavior and dynamic difficulty adjustment.

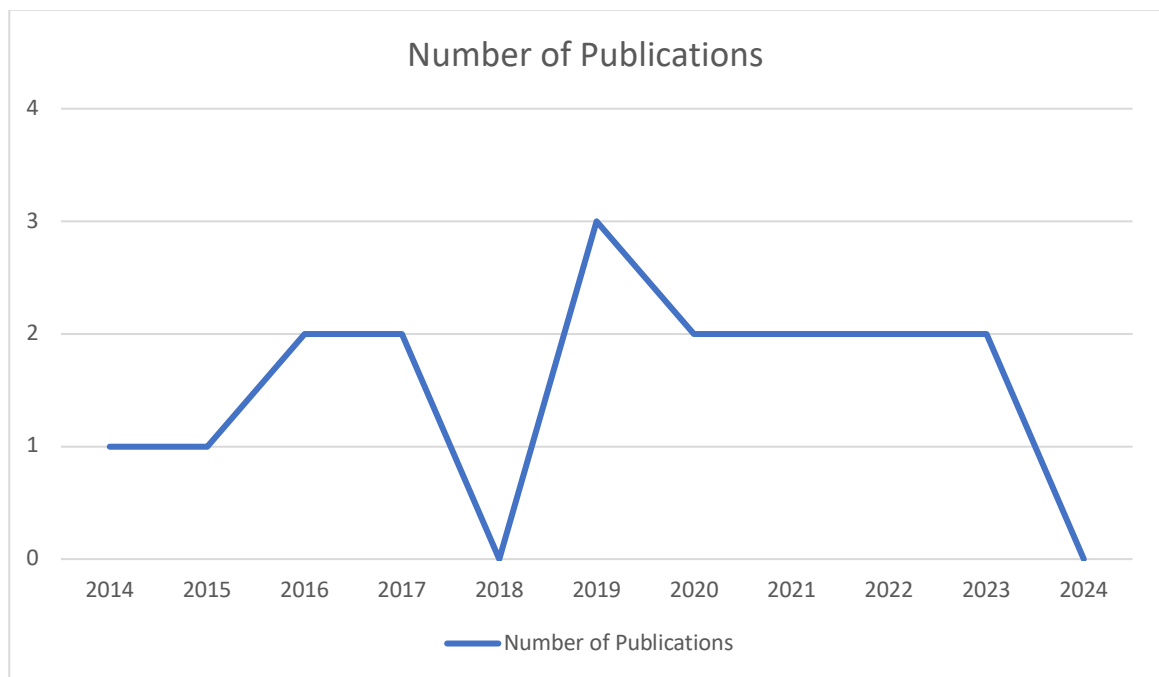


Figure 6 The number of journal publications

Figure 6 shows the number of related publications within 2014 to 2024. The biggest number of publications is 2019. Both 2018 and 2024 have no publications be selected so far.

Table 2 Total Number of Publications by Research Topic

Research Topic	Number of Publications
Dynamic Difficulty Adjustment (DDA)	7
NPC behavior	10

Among the literature included in this review, research on NPC behavior is more abundant, with a larger number of articles than that on dynamic difficulty adjustment (Table 2). This result indicates that NPC behavior plays a significant role in game design and development and may become a hot topic for future research.

The Table 3 illustrates the popularity of various AI techniques in 17 publications. Reinforcement Learning is the most frequently used technique, appearing in 9 publications. Deep Learning, Neural Networks, Behavior Trees, Finite State Machines and OpenAI Five

are each featured in 2 publications. Other techniques such as Hidden Markov Model, AI Director, AlphaGO, and Alphastar are each mentioned in 1 publication. Notably, the total number of publications exceeds the number of techniques, suggesting that some publications utilize multiple approaches.

Table 3 The Number of Publications by Techniques

Techniques	The Number of Related Publications
Deep Learning	2
Reinforcement Learning	9
Neural networks	2
Hidden Markov Model	1
Behavior Trees	2
Finite State Machines	2
OpenAI Five	2
AI Director	1
AlphaGO	1
Alphastar	1

In section 5 and section 6, these articles will be combined to discuss the specific application of AI in NPC behavior and DDA.

5 AI Applications in NPC Behavior

Traditional AI, often referred to as Game AI, relies on pre-defined rules and scripts to govern the behavior of non-player characters (NPCs) and game systems. These AI agents follow predetermined patterns and strategies, offering a predictable challenge to players.

Real AI, on the other hand, leverages advanced techniques such as machine learning and neural networks to enable more autonomous and adaptive decision-making. Real AI agents can learn from experience, adapt to changing circumstances, and exhibit behaviors that are

less predictable and more human-like. This can lead to more dynamic and engaging gameplay experiences.

5.1 Real AI in NPCs Behaviors

The advent of AI has revolutionized NPC behavior modeling. Machine learning techniques, such as decision trees, Bayesian networks, and neural networks, have been employed to create NPCs that can learn and adapt to their environments. These methods enable NPCs to make decisions based on perceived information, leading to more dynamic and unpredictable behaviors. Recent research has focused on incorporating more sophisticated AI techniques into NPC behavior modeling. This includes the use of natural language processing (NLP) to create NPCs capable of engaging in meaningful conversations with players. Additionally, emotion modeling and personality systems are being integrated to create NPCs with believable emotional responses and distinct personalities.

Despite significant progress, several challenges remain in NPC behavior modeling. Creating NPCs that are truly believable and capable of complex social interactions remains a difficult task. Moreover, balancing the autonomy of AI-driven NPCs with the designer's intended narrative can be challenging.

Real AI, unlike traditional AI, is designed to learn and adapt to the game environment dynamically. Instead of relying on pre-programmed scripts, real AI utilizes advanced algorithms such as machine learning and deep reinforcement learning to improve its gameplay over time. This enables real AI to provide a much more challenging and engaging experience for players, as it continuously evolves and adjusts its strategies based on the player's actions and the changing game state. Real AI also strives for fairness and transparency in its interactions with players. It doesn't resort to cheating or exploiting loopholes to gain an unfair advantage. Instead, it competes with players on an equal footing, relying on its learned skills and decision-making abilities to make the game more enjoyable and immersive.

5.1.1 Real AI vs Traditional AI

With the advancement of technology, real AI has shown significant changes and improvements compared to traditional AI in gaming, becoming increasingly similar to real players. While traditional AI relies on predefined rules and heuristics, real AI like utilizes computer vision, deep learning, and reinforcement learning to understand game visuals, make intelligent decisions, and adapt to complex and dynamic game environments.

First, real AI can observe and understand games like a real player. It utilizes advanced computer vision techniques, such as deep neural networks, to extract and analyze features from game visuals, surpassing the limitations of traditional AI's reliance on simple rules or pixel-level processing. This enables real AI to understand the game environment more accurately, even capturing subtle details in game visuals, leading to judgments that align more closely with human players' intuition.

Second, real AI's decision-making and actions are becoming increasingly similar to those of real players. Through reinforcement learning, real AI can learn from vast gaming experiences, adjusting strategies dynamically based on the current game state and continuously optimizing decisions, just like human players. In contrast, traditional AI often relies on fixed strategies or search algorithms, lacking this flexibility and adaptability.

Additionally, real AI exhibits growing adaptability and creativity, similar to real players. It can adapt to new game environments through continuous learning and practice, and even create new game strategies and behaviors, mirroring the learning process of human players. Traditional AI, on the other hand, often struggles in unfamiliar situations.

Real AI's advantages over traditional AI lie in its higher level of intelligence and adaptability. It leverages advanced algorithms like deep learning to learn from large-scale data and continuously optimize decision-making processes, adapting to complex game environments and player behaviors. Traditional AI, relying on pre-programmed rules and heuristic methods, may perform well in certain areas but falls short in complex and dynamic scenarios.

However, traditional AI still holds advantages in terms of simplicity, interpretability, lower computational costs, and less demand for hardware resources. In specific tasks and domains, traditional AI remains a viable option.

In conclusion, real AI has demonstrated superior performance in understanding game visuals, intelligent decision-making, adaptability, and creativity, increasingly resembling the behavior of real players. It outperforms traditional AI in complex gaming environments due to its advanced algorithms and learning capabilities. However, the choice between real AI and traditional AI still depends on the specific task, available resources, and desired level of intelligence and adaptability.

5.1.2 Deep Q-network (DQN)

One notable advancement is the use of deep reinforcement learning (DRL), Mnih et al., 2015 use recent advances in training deep neural networks to develop a novel artificial agent, termed a deep Q-network, that can learn successful policies directly from high-dimensional sensory inputs using end-to-end reinforcement learning. It can directly learn successful strategies from high-dimensional sensory inputs, and its performance on the classic Atari 2600 games exceeds that of previous algorithms, reaching the level of professional human players.

The key features of DQN include:

- Using deep convolutional neural networks to approximate the optimal action-value function.
- Adopting experience replay and periodically updating the target network to address the instability issues in reinforcement learning.
- The learned representations can support adaptive behavior from high-dimensional sensory inputs to actions.

It performs well on various types of games, including shooting games, boxing games, and racing games. However, it still faces challenges in games that require longer-term planning.

The success of DQN demonstrates the potential of combining deep learning with reinforcement learning, taking an important step towards achieving general artificial intelligence

5.1.3 Alpha Go

The game of Go has long been viewed as the most challenging of classic games for artificial intelligence due to its enormous search space and the difficulty of evaluating board positions and moves (Silver, et al., 2016). Traditional AI typically uses search algorithms, including Minimax and Alpha-Beta pruning. In games like chess or Go, traditional AI attempts to find the best move by searching the game tree. It predicts the consequences of each possible move and selects the one that leads to the most favorable outcome. Minimax algorithm evaluates each possible move by recursively maximizing and minimizing the search on the game tree. Alpha-Beta pruning reduces the search space by pruning during the search process, making it more efficient to find the best move.

Silver, et al., (2016) propose the program AlphaGo, a Go program that combines deep neural networks and Monte Carlo tree search algorithms. It uses two neural networks to evaluate the game position: one is the policy network, which is used to predict the best next move; the other is the value network, which is used to predict the final outcome of the game. In the search process, AlphaGo starts from the root node and simulates the game, and selects the next move based on the output probability of the policy network and the previous visit count. When it reaches the leaf nodes, it uses the value network and the results of random simulations to evaluate the value of the leaf nodes. This combination of the policy network and the value network allows AlphaGo to make strong playing decisions with relatively few simulations. Compared to previous Go programs, AlphaGo intelligently selects the game positions to evaluate, and uses more powerful neural networks to evaluate the positions, making up for the disadvantage of fewer simulations. Through this approach, AlphaGo ultimately defeated human professional Go players, achieving a major breakthrough in artificial intelligence in the field of Go. In 2017, AlphaGo defeated the world's top Go player Jie Ke in a five-game match.



Figure 7 JIE KE vs AlphaGO

5.1.4 Alpha Star

After defeating humans in chess and Go, the next target for AI was to defeat masters in real-time strategy game StarCraft II. Both the original game and its sequel, StarCraft II, present more challenges than Go due to features such as real-time gameplay, partial observability, absence of a single dominant strategy, complex rules, difficulty in establishing fast-forward models, and an exceptionally large and diverse action space (Arulkumaran, et al., 2019). The projects like AlphaStar and OpenAI Five, represents a significant advancement in the field of artificial intelligence in gaming.

AlphaStar, developed by DeepMind, employs deep neural networks and reinforcement learning to master the complex real-time strategy game "StarCraft II." AlphaStar draws on many areas of AI research, including deep learning, reinforcement learning, game theory, and evolutionary computation (EC) (Arulkumaran, et al., 2019). The neural network model of AlphaStar is designed to take raw image inputs from the game and output probability distributions over actions. Through training on billions of self-play games, AlphaStar continuously refines its strategies, gradually mastering advanced tactics and micro-

management skills, ultimately demonstrating outstanding performance in battles against professional human players.

5.1.5 OpenAI Five

OpenAI Five is a project developed by OpenAI team that showcases the capabilities of artificial intelligence (AI) in playing the complex multiplayer online battle arena (MOBA) game Dota 2. It represents a significant milestone in AI research, demonstrating that AI systems can learn to play at a high level in a game that requires strategic thinking, teamwork, and quick decision-making.

Berner, et al., (2019) introduce three challenges that OpenAI Five needs to solve when playing Dota2:

- Long time horizons
- Partially-observed state.
- High-dimensional action and observation spaces.

Additionally, there are two limitations for the OpenAI Five:

- Only 17 heroes can be played, while the hero pool in the game is 117.
- Items that require controlling multiple characters are not supported

OpenAI Five consists of five independent but collaborative AI agents, each controlling a game character (hero). These agents work as a team by sharing information. It uses reinforcement learning for training, improving its strategies and skills through a large number of simulated gameplays. By reading multidimensional game state data such as hero positions, health, mana, items, skill statuses, and map information, the AI employs deep learning models to extract critical features for an accurate understanding of the game environment.

In terms of decision-making, OpenAI Five relies on policy networks and value networks. The policy network predicts the optimal action at each timestep, including movements, attacks, and the use of skills and items, while the value network assesses the current state's

value to help the policy network make better decisions. By calling the game engine's API, the AI agents can execute specific game operations within milliseconds, ensuring real-time responsiveness to in-game changes.

They developed a distributed training system and continuous training tools to train OpenAI Five for 10 months. This training system utilized deep reinforcement learning techniques and proximal policy optimization algorithms. On April 13th, 2019, the OpenAI Five defeat the world Dota 2 world champions (Team OG), it demonstrated that self-play reinforcement learning can achieve superhuman performance on a difficult task (Berner, et al., 2019).

6 Dynamic Difficulty Adjustment

Dynamic Difficulty Adjustment (DDA) is a common approach for optimizing player experience by automatically modifying game aspects (Mortazavi, et al., 2024). It consists of automatically changing parameters, scenarios, and opponent's actions and/or behaviors in a video game in real-time, based on the player's skill, to avoid making the player bored or frustrated (Climent, et al., 2024). This technique enhances the game's playability, allowing a wider range of players to enjoy the game and providing a smoother gaming experience. The balance between game difficulty and player skill in the evolving landscape of the video game industry is a significant factor in player engagement (Romero-Mendez, et al., 2023).

Before the application of DDA, the game used static difficulty. Players could choose their favorite difficulty through preset options, such as easy, normal, and difficult. However, this primitive approach has some fundamental flaws: the number of different difficulty levels is limited, the differences between levels may be too steep for some players, and players are more susceptible to changes in experience. Although some games allow players to reselect difficulty throughout the game, this is at the expense of the player's gaming experience and destroys the integrity of the gaming experience.

The basic principle of DDA is to monitor the player's behavior and game performance to dynamically adjust the game's difficulty. This typically involves collecting gameplay data

such as progress, success rate, failure count, etc., and then adjusting various parameters of the game, such as enemy health, spawn rates, level design, etc., based on this data.

The game Left 4 Dead (by Valve Corporation) employ an AI called the "Director (sometimes called the AI Director, or simply AID)", which dynamically adjusts enemy spawns, item placements, and combat intensity based on player performance and decisions, ensuring a consistently challenging and tense gaming experience. The AI Director decides where weapons, ammo and useful items are spawned, and decides when and where a desperate horde of angry zombies (an infected rush) starts (Agis, et al., 2016) .

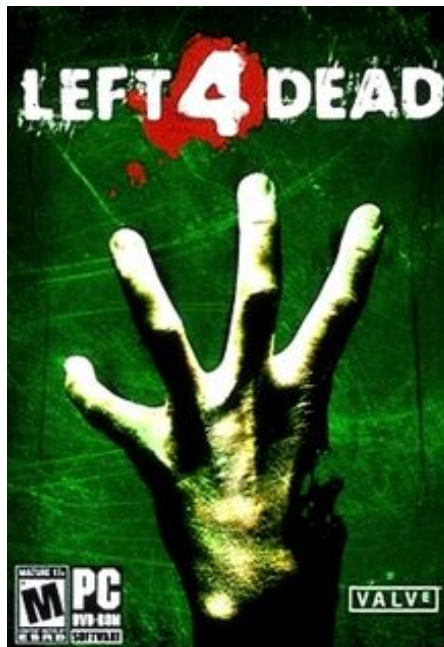


Figure 8 Cover of Left 4 Dead

The dynamic difficulty adjustment system in Resident Evil 4 (by Capcom) adjusts the game difficulty based on the player's initial selection and performance, primarily considering the player's death count. If the player continually died and struggled through a section, the system will decrease the game's difficulty, for example, reducing the number and damage of enemies, and increasing the quantity of items and weapons the player can obtain from crates. Conversely, if a player performs well, the system will create more enemies and increase their damage.

Table 4 Item Distribution by Player Ranking in Mario Kart 8 Deluxe

Ranking Items	1st Place Player	Mid-Ranking Players	Bom-Ranking Players
Coin	✓	✓	✓
Green Shell	✓	✓	✓
Triple Green Shells	×	✓	✓
Red Shell	×	✓	✓
Triple Red Shells	×	✓	✓
Banana	✓	✓	✓
Triple Bananas	×	✓	✓
Mushroom	×	✓	✓
Triple Mushrooms	×	✓	✓
Golden Mushroom	×	×	✓
Bob-omb	✓	✓	✓
Super Horn	✓	✓	✓
Blooper	×	✓	✓
Fire Flower	×	✓	✓
Boomerang Flower	×	✓	✓
Piranha Plant	×	✓	✓
Spiny Shell	×	×	✓
Super Star	×	×	✓
Bullet Bill	×	×	✓
Lighting Bolt	×	×	✓
Boo	×	×	✓
Crazy 8	×	×	✓

In Nintendo's Mario Kart 8 Deluxe, one of the most apparent dynamic difficulty adjustment systems is the item distribution system. Based on players' rankings, the game allocates different types and strengths of items to enhance the game's balance and challenge. Lower-ranked players often receive more powerful items to help them catch up with the leaders, while higher-ranked players may receive relatively weaker items to maintain the race's balance. This item distribution system creates a dynamic and challenging gaming experience, providing players with a fair and exhilarating racing environment. The table 1 illustrates the item distribution in Mario Kart 8 Deluxe based on player rankings.

Adaptive game difficulty aims to dynamically adjust the game's challenge level according to the player's skill level, to maintain an appropriate experience. Hunicke and Chapman (2004) propose Hamlet, a probabilistic technique to dynamically evaluates the difficulty of given obstacles based on user performance, aiming to maintain player engagement within the flow channel by fostering specific states and discouraging others, thereby optimizing interaction loops based on individual skill level and game-specific experience.

Yang. j et al.(2009) proposes a method to create intelligent adaptive game opponents using the Monte-Carlo algorithm. The Monte-Carlo algorithm was applied to the Dead End game to improve the adaptability of the opponent's AI by building a search tree. At the same time, the study also explores the combination of artificial neural networks with Monte-Carlo to improve the intelligent adaptability of game opponents.

The advent of machine learning has opened up new avenues for DDA research. Researchers have explored various algorithms, such as Bayesian networks, decision trees, and neural networks, to model player behavior and predict their skill level. For example, Jennings-Teats, et al.(2010) propose Polymorph, a model uses machine learning technology to dynamically generate levels suitable for the player's skill level. It learns the difficulty characteristics of levels by collecting large-scale player behavior data on short level segments, and establishes a difficulty model. During the game process, Polymorph will adjust the difficulty of subsequent levels in real-time based on the player's performance, creating a personalized gaming experience for the player. This machine learning-based dynamic level generation method can be applied to other game types to provide players with a better gaming experience.

Reinforcement learning (RL) has emerged as a promising approach for DDA in recent years. RL algorithms learn by interacting with the environment and receiving feedback in the form of rewards. For instance, Sekhvat (2017) adopts a personalized difficulty adjustment technique called "Multiple-Periodic Reinforcement Learning (MPRL)" for automatic difficulty adjustment in rehabilitation games. The MPRL method uses reinforcement learning techniques, allowing the definition of multiple time periods to evaluate different objectives, and associating each objective with a different set of states, thereby better handling potential conflicts between objectives. Through this design, the MPRL method achieves automatic personalized adjustment of game difficulty, improving patient engagement and rehabilitation outcomes.

Although DDA research has made significant strides, challenges persist in finding the optimal balance between challenge and enjoyment due to varying player preferences. Additionally, ensuring that DDA systems feel natural and not manipulative to players remains a concern. Furthermore, ethical considerations regarding the collection and use of

player data, as well as potential biases in AI algorithms, need to be thoroughly addressed. Future research in DDA is expected to concentrate on developing more advanced AI models that can learn from a wider range of player information, including biometrics, emotions, and social interactions.

6.1 Hidden Markov Models

Hidden Markov models are a general statistical modelling technique for 'linear' problems like sequences or time series and have been widely used in speech recognition applications for twenty years (Eddy, 1996). Moreover, HMMs are also rich enough to handle real world application. (Rubayyi, 2016). The mathematical structure of HMM makes the theoretical basis for many real-world applications like speech recognition, facial expression recognition, gene prediction, gesture recognition, musical composition and Bio-informatics (Bhavya, et al., 2021)

Hidden Markov works based on double random processes: one type of sequence data can be observed, that is, the observation sequence; while the other type of data cannot be observed, that is, the hidden state sequence, referred to as the state sequence. HMM infers hidden parts based on the correlation of observable observation processes. For example, when I am typing and writing a thesis, the series of characters I type on the keyboard is the observation sequence, and the paragraph I want to write is the hidden sequence. The task of the input method is to extract as much information as possible from the series of characters I type and guess the paragraph I want to write and put the most likely words at the front for me to choose. This can be regarded as an HMM model.

Zamith et al. (2020) propose a new method for dynamically adjusting difficulty levels using machine learning techniques based on Hidden Markov Models (HMMs). The researchers built a game similar to the Space Invaders and used the Hidden Markov Model (HMM) module to predict player movement for dynamic difficulty adjustment. The main objective of the game is to destroy the enemy by shooting while avoiding the enemy's attacks. The purpose of the HMM module is to learn the player style of each n player moves and adjust the difficulty of the game accordingly.

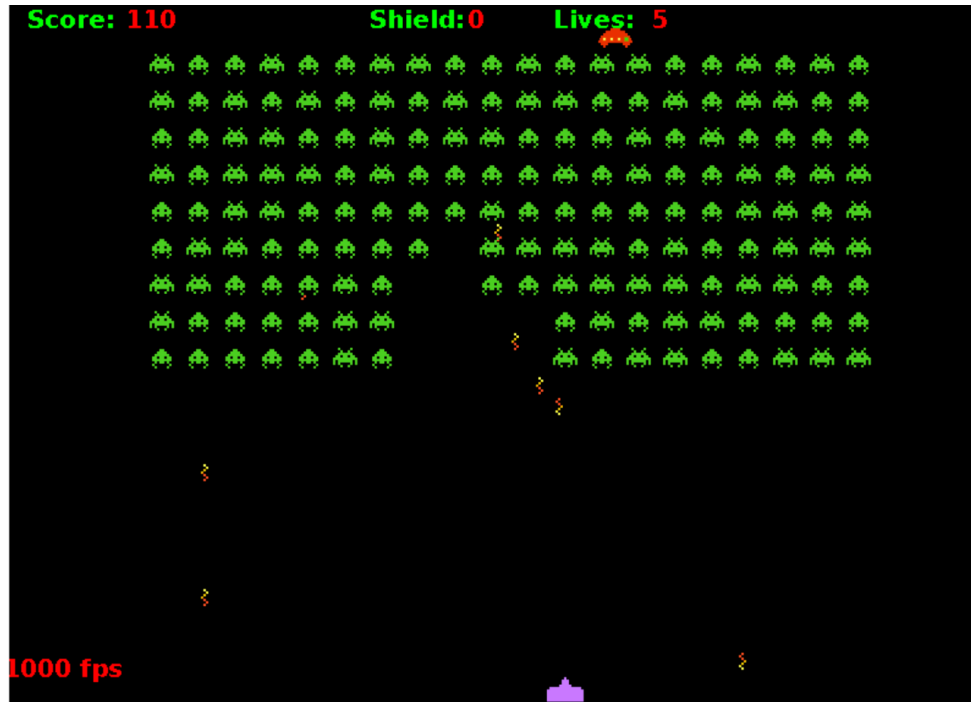


Figure 9 Space Invader by Zamith et al

In this study, the observable sequence consists of the player's movements, which include three possible actions: moving left, moving right, and stopping. The hidden sequence represents the state of the enemy spacecraft, specifically the probability transition matrix that determines the likelihood of the enemy spacecraft being destroyed. Each enemies having a probability p of being destroyed based on the player's skills and positions. The game collects the player's movement data in real-time to learn the player's behavior patterns. Based on these learned patterns, the game dynamically adjusts the enemy's defense strategies, such as turn on their force shields and increase the frequency of shoots for 10 seconds (Zamith, et al., 2020), to enhance the game's dynamic difficulty. This HMM module updates every 100 player movements to adapt to changes in player skills, ensuring the calculation time remains within a game frame to maintain interactivity.

The calculation process of HMM can be summarized as three steps:

- Initialization: Allocate and initialize the Transition probability matrix A , Observation probability matrix B (calculated based on the number of enemies), Initial probability vector π of the HMM

- Training: Apply the Baum-Welch algorithm to learn and update the transition probability matrix A and the initial probability vector π , thereby constructing the complete HMM model $\lambda = (A, B, \pi)$.

$$\text{Viterbi Algorithm : } a_{i,j} = \frac{E[S_{i,j}]}{E[S_i]}$$

- Most Likely State Sequence: Use the Viterbi algorithm to calculate the most likely state sequence of the hidden part of the HMM.

In these processes, the training step (Baum-Welch algorithm) occupies nearly 90% of the computation time, while the initialization and the calculation of the most likely state sequence are relatively fast. It should be noted that the computational complexity of the HMM algorithm grows exponentially with the increase in the number of hidden states (i.e., the number of enemies in the game). This may affect the interactivity of the game, so a balance between algorithm complexity and game performance needs to be considered.

This study verifies the feasibility and effectiveness of HMM application in personalized, real-time game difficulty adjustment. However, the research still has some limitations:

- Lack of description of specific difficulty adjustment methods
- The game observes the player's movement pattern and calculates the probability of the enemy being destroyed based on the distance and angle between the player and the enemy, and then adjusts the game difficulty based on this. Instead of based on the player's real strength, such as the hit rate of shooting. This will bring a new gameplay experience to the players, rather than providing the appropriate difficulty based on the player's strength.
- The control group game without HMM did not specify whether there is a "force shield" and "increase shooting frequency" mechanism, and if so, what is its triggering mechanism.

To overcome the limitations identified in the study, there are some improvements:

- **Personalized Difficulty Adjustment:** Future research could explore integrating multiple player performance indicators to achieve more personalized game difficulty adjustments, thereby providing a more intelligent and immersive gaming experience.
- **Consideration of Player Skill:** It is recommended to incorporate player skill directly into the adjustment model and explore methods for personalizing game difficulty to provide a gaming experience that better aligns with player expectations.
- **Rigorous Control Group Design:** Future studies should meticulously design control group experiments and further investigate the impact of different game mechanisms to more accurately evaluate the effectiveness of the HMM model in game difficulty adjustment.

Nevertheless, this study still provides new ideas and inspiration for dynamic game balance, which has important academic value and practical significance. Future research can be further improved by expanding the scope of experiments, detailing the difficulty adjustment method.

6.2 Reinforcement learning

Reinforcement Learning (RL) is a machine learning paradigm aimed at enabling an agent to learn how to take actions in an environment to maximize cumulative rewards. It learns what to do—how to map situations to actions—so as to maximize a numerical reward signal (Sutton & Barto, 2020). Essentially, Reinforcement Learning (RL) is the process of mapping the states of an agent (also called situations) to a set of actions (Sekhavat, 2017).

In RL, an agent interacts with an environment by observing its states, taking actions, and receiving rewards or penalties from the environment to learn how to choose actions. The core idea is to learn through trial and error to maximize long-term returns.

Key Components of Reinforcement Learning:

- Agent: The entity that interacts with the environment by taking actions. Its goal is to learn the best policy to maximize cumulative rewards.
- Environment: The external world where the agent operates. Its state may change as the agent takes actions and generates corresponding rewards or penalties.
- State: A specific condition or configuration of the environment used to describe its current situation.
- Action: The behavior or decision that the agent can take in a specific state.
- Reward: Defines the goal of a reinforcement learning problem (Sutton & Barto, 2020). Agent will get feedback from the environment after an action is taken, used to evaluate the action's quality.
- Policy: Defines how the agent chooses actions in specific states. Roughly speaking, a policy is a mapping from perceived states of the environment to actions to be taken when in those states (Sutton & Barto, 2020).
- Value Function: Evaluates the expected long-term return in specific states or state-action pairs, helping the agent decide which actions to choose.
- Learning Algorithm: Updates the agent's policy or value function to iteratively improve the agent's performance.

Cherukuri & Glavin (2022) proposes a dynamic difficulty adjustment (DDA) technique called "Skilled Experience Catalogue (SEC)" for application in fighting games. They tested their DDA system using FightingICE (FTG), a fighting game developed by the Intelligent Computer Entertainment Lab (ICE Lab) at Ristumeikan University. The SEC technique utilizes the experience milestones in the learning process of reinforcement learning (RL) agents to dynamically adjust the game difficulty to match the player's skill level. This method can effectively balance the game difficulty, improve player engagement and experience, but is limited by the highest skill level that the underlying learning algorithm can achieve.

Furthermore, if a deep reinforcement learning algorithm is adopted, it may result in a larger model size, leading to noticeable latency during difficulty adjustment. Overall, the SEC technique is a promising dynamic difficulty adjustment method but requires further optimization to improve its performance and applicability.

In same platform, De Oliveira & Chaimowicz (2023) designed a RL-based DDA system with the goal of keeping the relative skill difference between the player and the opponent within a balanced range. They introduce Balance Reward mechanism and Competitiveness Penalty mechanism aimed at dynamic difficulty adjustment in games using reinforcement learning. These mechanisms are designed to maintain game balance and challenge to provide a satisfying gaming experience.

Balance Reward is a reward function intended to keep the relative skill difference between players ($\Delta Skillt(p1, p2)$) within a balanced range ($IBC = [-BC/2, BC/2]$). The reward function employs a symmetric square function, providing positive rewards when the relative skill difference is within the balance range and negative rewards when it exceeds the range. This helps guide the agent to achieve a relative skill level comparable to the player, avoiding gameplay that is either too easy or too difficult.

Competitiveness Penalty a penalty mechanism aimed at limiting the agent's win rate against players to maintain reasonable challenge levels. The authors propose a competitiveness interval ($Ic = [pi, pj]$) to restrict the agent's win rate within a reasonable range. This ensures that the agent is neither too dominant to provide challenge nor too weak to offer an engaging gaming experience.

In summary, this research proposes a novel method based on Balance Reward and Competitiveness Penalty to dynamically adjust game difficulty, aiming to balance player win rates and enhance gaming experience. Through experiments, the effectiveness of this method is validated across opponents of varying difficulty levels, with the ability to restore balance even in initially imbalanced states. Additionally, the study explores the effects of different regularization techniques on improving agent generalization capabilities.

However, limitations are identified, including slow convergence of agents against opponents of moderate difficulty and the inability of agents to fully converge to the target competition interval even with regularization techniques. Furthermore, the study primarily focuses on relative opponent skill levels, while differences in opponent

behavior styles may also impact game balance, warranting further investigation. Finally, the experiments are conducted in simulated environments, necessitating further evaluation of the method's effectiveness and player experience in real-player settings.

7 DISCUSSION AND FUTURE WORK

NPC behavior and Dynamic Difficulty Adjustment (DDA) have significant complementarity and synergy in improving the gaming experience. The intelligence of NPC behavior enhances immersion and interaction in games, making players feel a more realistic and engaging virtual world. DDA adjusts the game's difficulty in real-time, ensuring players are always at an appropriate challenge level, avoiding boredom or frustration caused by either overly simple or difficult gameplay.

Integrating NPC behavior and DDA technologies can achieve a more personalized and dynamic gaming experience. For example, in action-adventure games, intelligent NPCs can react according to the player's strategies and actions, while DDA systems adjust NPC difficulty based on player performance, providing a challenging yet not overly difficult gameplay experience. This synergy not only enhances the game's replayability and longevity but also increases player satisfaction and engagement.

7.1 Research Gaps and Challenges

Despite significant progress in NPC behavior and DDA, there are still some research gaps and challenges.

- **Data Requirements:** Training intelligent NPCs and DDA systems typically requires large amounts of data, including game data and player behavior data. Acquiring and processing this data may pose challenges, especially considering game content and player privacy.

- Computational resources: Implementing complex AI algorithms requires significant computational resources, including the ability to handle large-scale data and run complex models. This may limit smaller development teams or resource-constrained developers.
- Model generalization: Different types of games and player behavior patterns vary widely. Developing AI models with good generalization capabilities to adapt to various game scenarios and player demographics is a significant challenge.
- Real-time responsiveness and latency: In actual gameplay, AI systems need to respond in real-time to player actions and performance. Balancing real-time responsiveness with accurate and efficient dynamic adjustments is a technical challenge.
- Player privacy and data security: Collecting and analyzing player behavior data raise privacy and data security concerns. Balancing personalized experiences with player privacy and data security requires further research and solutions.
- Stability: Stable NPC behavior is essential for providing a consistent gaming experience. Ensuring the stability of AI NPC behavior requires careful tuning and testing to avoid confusion or poor gaming experiences caused by unstable behavior.
- Balancing NPC Autonomy with Design Intent: While granting AI-driven NPCs greater autonomy, ensuring their actions align with the game designer's intent and avoid disrupting the game narrative and experience is a delicate balancing act.
- Long-term Maintenance: Games undergo long-term updates and maintenance, including adding new content, fixing bugs, and optimizing performance. Therefore, AI NPC and DDA systems need to adapt continuously to game changes without causing instability or performance degradation due to updates.

7.2 Future Research

Future research can improve and explore the following aspects:

- Cross-platform and cross-game AI models: Developing general AI models that can be applied across multiple game types and platforms to improve model generalization and adaptability.

- Hybrid AI technologies: Combining various AI technologies, such as integrating deep learning with traditional Finite State Machines (FSM) or Behavior Trees, to leverage their respective strengths and enhance NPC behavior and DDA effectiveness.
- Data privacy protection technologies: Researching effective ways to protect player privacy and data security during the collection and analysis of player data, building secure and reliable game data management systems.

8 Conclusion

The application of Artificial Intelligence (AI) in the gaming industry is revolutionizing the way players interact with and experience games. This paper explores the use of AI in enhancing Non-Player Character (NPC) behavior and implementing Dynamic Difficulty Adjustment (DDA) to improve the overall gaming experience. From early expert systems to current deep reinforcement learning models, AI technology has continuously evolved in game design.

Machine learning techniques, such as decision trees, Bayesian networks, and neural networks, have been employed to create NPCs that can learn and adapt to their environments. These methods enable NPCs to make decisions based on perceived information, leading to more dynamic and unpredictable behaviors. Recent research has focused on incorporating more sophisticated AI techniques into NPC behavior modeling, including the use of natural language processing (NLP) to create NPCs capable of engaging in meaningful conversations with players, and the integration of emotion modeling and personality systems to create NPCs with believable emotional responses and distinct personalities.

DDA techniques aim to dynamically adjust game difficulty based on the player's skill level and gaming experience, to maintain an appropriate level of challenge and enhance enjoyment. This can be achieved through various methods, including Bayesian networks, decision trees, neural networks, and reinforcement learning algorithms. These techniques have been applied in various game genres, such as action-adventure games, fighting games, and racing games, to provide more engaging and personalized gaming experiences.

Despite significant progress in NPC behavior and DDA, challenges remain, such as data requirements, computational resources, model generalization, real-time responsiveness and latency, player privacy and data security, stability, and balancing NPC autonomy with design intent. Future research can focus on developing more advanced AI models that can learn from a wider range of player information, including biometrics, emotions, and social interactions. Additionally, the development of cross-platform and cross-game AI models, hybrid AI technologies, and data privacy protection technologies presents promising avenues for further enhancing the application of AI in gaming.

In conclusion, the application of AI technology in improving the gaming experience not only enhances gameplay and interaction but also opens up new directions and possibilities for future game development. Through continuous research and innovation, AI is poised to play an increasingly important role in the gaming industry, offering players richer and more personalized gaming experiences.

References

- Agis, R. A., Cohen, A. & Martínez, D. C., 2016. *Argumentative AI Director Using Defeasible Logic Programming*. Cham: Springer International Publishing .
- Arulkumaran, K., Antoine, C. & Julian, T., 2019. Alphastar: An evolutionary computation perspective. *s.l., s.n.*, pp. 314-315.
- Berner, C. et al., 2019. Dota 2 with large scale deep reinforcement learning.. *s.l.*, arXiv preprint.
- Bhavya, M., Sunita, G. & Ajay, K., 2021. A Systematic Review of Hidden Markov Models and Their Applications. *Archives of Computational Methods in Engineering*, 5, 28(3), pp. 1429-1448.
- Cherukuri, A. & Glavin, F. G., 2022. Balancing the Performance of a FightingICE Agent using Reinforcement Learning and Skilled Experience Catalogue. *St. Michael, Barbados, IEEE*, pp. 1-6.
- Climent, L., Longhi, A., Arbelaez, A. & Mancini, M., 2024. A framework for designing Reinforcement Learning agents with Dynamic Difficulty Adjustment in single-player action video games. *Entertainment Computing*, 5, p. 100686.
- Colledanchise, M. & Ogren, P., 2014. How Behavior Trees modularize robustness and safety in hybrid systems. *Chicago, IL, USA, IEEE*, pp. 1482-1488.
- Csikszentmihalyi, M., 2008. *The Psychology of Optimal Experience*. 1st Harper Perennial modern classics ed. New York: Harper Perennial.
- De Oliveira, T. N. & Chaimowicz, L., 2023. Investigating Reinforcement Learning for Dynamic Difficulty Adjustment. *s.l., ACM*, pp. 66-75.
- Eddy, S. R., 1996. Hidden Markov models. *Current Opinion in Structural Biology*, Volume 6, pp. 361-365.
- Hunicke, R. & Chapman, V., 2004. *AI for Dynamic Difficulty Adjustment in Games*. *s.l., Challenges in game artificial intelligence AAAI* .
- Hunicke, R., LeBlanc, M. & Zubek, R., 2004. MDA: A Formal Approach to Game Design and Game Research. *Proceedings of the AAAI Workshop on Challenges in Game AI*, 4(1).
- Jagdale, D., 2021. Finite State Machine in Game Development. *International Journal of Advanced Research in Science, Communication and Technology*, pp. 384-390.

- Jennings-Teats, M., Smith, G. & Wardrip-Fruin, N., 2010. Polymorph: A Model for Dynamic Level Generation. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 10 10, 6(1), pp. 138-143.
- Kirby, N., 2011. *Introduction to game AI*. Boston, Mass: Cengage Learning.
- Koster, R., 2013. *Theory of Fun for Game Design*. 2nd ed. s.l.:O'Reilly Media, Incorporated.
- Millington, I., 2019. *AI for Games*. 3rd ed. Boca Raton: CRC Press.
- Mnih, V. et al., 2015. Human-level control through deep reinforcement learning. *Nature*, 26 02, pp. 529-533.
- Mortazavi, F., Moradi, H. & Vahabie, A.-H., 2024. Dynamic difficulty adjustment approaches in video games: a systematic literature review. *Multimedia Tools and Applications*, 12 3.
- Newton, P. L. & Feng, J., 2016. *Unreal Engine 4 AI programming essentials: create responsive and intelligent game AI using Blueprints in Unreal Engine 4*. Birmingham, UK: Packt Publishing.
- Romero-Mendez, E. A. et al., 2023. The Use of Deep Learning to Improve Player Engagement in a Video Game through a Dynamic Difficulty Adjustment Based on Skills Classification. *Applied Sciences*, 2023-07-16 07, 13(14), p. 8249.
- Rubayyi, A., 2016. Hidden Markov Models (HMMs) and Security Applications. *International Journal of Advanced Computer Science and Applications*, 7(2).
- Sekhavat, Y. A., 2017. Behavior Trees for Computer Games. *International Journal on Artificial Intelligence Tools*, 04/2017 04, 26(02), p. 1730001.
- Sekhavat, Y. A., 2017. MPRL: Multiple-Periodic Reinforcement Learning for difficulty adjustment in rehabilitation games. Perth, Australia, IEEE, pp. 1-7.
- Silver, D. et al., 2016. Mastering the game of Go with deep neural networks and tree search. *Nature*, 28 1, pp. 484-489.
- Srisuphab, A. & Silapachote, P., 2013. Rule-based systems made easy with battleship games: A well-received classroom experience. Bali, Indonesia, s.n.
- Sutton, R. S. & Barto, A., 2020. *Reinforcement learning: an introduction*. Second edition ed. Cambridge(Massachusetts London): The MIT Press.
- U, S. K. et al., 2021. Artificial Intelligence (AI) Prediction of Atari Game Strategy by using Reinforcement Learning Algorithms. Shillong, India, IEEE, pp. 536-539.
- Yang, J. et al., 2009. To Create Intelligent Adaptive Game Opponent by Using Monte-Carlo for Tree Search. Tianjian, China, IEEE, pp. 603-607.

Yannakakis, G. N. & Togelius, J., 2018. Artificial Intelligence and Games. Cham: Springer International Publishing.

Zamith, M., da Silva Junior, J. R., Joselli, M. & Clua, E. W. G., 2020. Applying Hidden Markov Model for Dynamic Game Balancing. Recife, Brazil, 19th Brazilian Symposium on Computer Games and Digital Entertainment (SBGames), pp. 38-46.

Appendix 1. Table of the selected papers.

Paper	Year	Research Topic	AI Techniques
Human-level control through deep reinforcement learning	2015	NPC Behavior	Deep Learning, Reinforcement Learning
Mastering the game of Go with deep neural networks and tree search	2016	NPC Behavior	Neural Networks, AlphaGo
Alphastar: An evolutionary computation perspective	2019	NPC Behavior	Alphastar, Reinforcement Learning, Neural Networks
MPRL: Multiple-Periodic Reinforcement Learning for difficulty adjustment in rehabilitation games	2017	DDA	Reinforcement Learning
Applying Hidden Markov Model for Dynamic Game Balancing	2020	DDA	Hidden Markov Model
Investigating Reinforcement Learning for Dynamic Difficulty Adjustment	2023	DDA	Reinforcement Learning
Finite State Machine in Game Development	2021	NPC Behavior	Finite State Machine
Behavior Trees for Computer Games	2017	NPC Behavior	Behavior Trees

Appendix 1. Table of the selected papers. (continues)

Paper	Year	Research Topic	AI Techniques
Artificial Intelligence (AI) Prediction of Atari Game Strategy by using Reinforcement Learning Algorithms	2021	NPC Behavior	Reinforcement Learning
How Behavior Trees modularize robustness and safety in hybrid systems	2014	NPC Behavior	Behavior Trees
Balancing the Performance of a FightingICE Agent using Reinforcement Learning and Skilled Experience Catalogue	2022	DDA	Reinforcement Learning
The Use of Deep Learning to Improve Player Engagement in a Video Game through a Dynamic Difficulty Adjustment Based on Skills Classification	2023	DDA	Deep Learning
Argumentative AI Director Using Defeasible Logic Programming	2016	DDA	AI Director
Literature Review of OpenAI Five's Mechanisms in Dota 2's Bot Player	2022	NPC Behavior	OpenAI Five, Reinforcement learning

Appendix 1. Table of the selected papers. (continues)

Paper	Year	Research Topic	AI Techniques
Dota 2 with Large Scale Deep Reinforcement Learning	2019	NPC Behavior	OpenAI Five, Reinforcement Learning
Implementing Adaptive Game Difficulty Balancing in Serious Games	2019	DDA	Reinforcement Learning
Finite State Machines for Building Believable Non-Playable Character in the Game of Khalid ibn Al-Walid	2020	NPC Behavior	Finite State Machines